

SQL no Databricks do zero ao dashboard

Guia completo para iniciantes: cada comando explicado, com a consulta escrita por inteiro e o resultado que você deve ver na tela.

SUMÁRIO

O que você vai encontrar

Leia na ordem se está começando agora. Se já conhece SQL, pule direto para o capítulo do comando que precisa.

CONTEXTO E FUNDAMENTOS

01	O desafio	A empresa, os diretores e a entrega do dia
02	Antes do SQL: os conceitos	Tabela, linha, coluna, tipo e chave
03	O que é SQL	Uma linguagem declarativa, quase em inglês
04	O ambiente: Databricks	Catálogo, schema, notebook e serverless
05	As quatro tabelas	O que tem dentro de cada uma
06	Anatomia de uma consulta	As sete cláusulas e a ordem de execução

OS COMANDOS, UM A UM

07	SELECT e FROM	O que tem nesta tabela?
08	Colunas calculadas e AS	Criando a receita
09	ORDER BY e LIMIT	Ordenar e ficar com o topo
10	WHERE: filtrando linhas	Operadores, IN, BETWEEN, LIKE e DISTINCT
11	Agregações	COUNT, SUM, AVG, MIN e MAX
12	GROUP BY	O mesmo cálculo por grupo
13	HAVING	WHERE × HAVING, de uma vez por todas
14	JOIN	Juntando tabelas e a pegadinha do INNER JOIN

RESPONDENDO AS DIRETORIAS

15	Diretoria de Clientes	Top 10 clientes e receita por estado
16	Diretoria de Pricing	Nosso preço contra o mercado

ALÉM DO BÁSICO

17	Bônus: CASE WHEN	O se/então/senão do SQL
----	------------------	-------------------------

18	Bônus: window functions	Ranking, percentual e comparação
19	Bônus: subconsulta e CTE	Organizando consultas grandes
20	Por que tabela e não arquivo	Schema, histórico e time travel
21	O dashboard	Uma página por diretoria

APOIO

22	Erros comuns	Como ler a mensagem do Databricks
23	Exercícios	Dezesseis perguntas para praticar

APÊNDICES

•	Respostas dos exercícios	As 16 consultas resolvidas
•	Colinha	Uma página para deixar aberta
•	Os próximos três dias	Aulas 2, 3 e 4

CAPÍTULO 1

O desafio

A empresa, os três diretores e o que você precisa responder hoje.

Você acabou de ser contratado como analista de dados de um **e-commerce brasileiro** que vende pelo site e em uma loja física. Antes mesmo de organizar qualquer coisa, três diretores já pedem respostas.

Este guia acompanha a **Aula 1 da Imersão Jornada de Dados**. Ele foi escrito para quem **nunca escreveu SQL**. Cada comando aparece três vezes: o que ele faz, a consulta escrita por inteiro e o resultado que você deve ver na tela. Leia com calma, no seu ritmo. Nada aqui precisa ser decorado.

A empresa em números



O período dos dados vai de **13/12/2025 a 11/01/2026**. São 11 categorias de produto, 2 canais de venda (site e loja física) e 4 concorrentes monitorados: Mercado Livre, Amazon, Magalu e Shopee.

Os três diretores

Quem	O que pede	Capítulo que responde
Diretor Comercial Vendas	“Quanto vendemos? Qual canal vende mais? Quais produtos e categorias puxam a receita?”	11 a 14
Diretora de Customer Success Clientes	“Quem são os 10 melhores clientes? De quais estados eles são?”	15
Diretor de Pricing Preços	“Estamos mais caros que a concorrência? Em quais produtos?”	16

O que você entrega no fim do dia

Um **dashboard publicado**, com uma página para cada diretoria. Os diretores abrem o link e veem os números sem precisar falar com você. É isso que separa um relatório de uma entrega de dados.

A FRASE DA IMERSÃO

No dia 1 você responde o diretor. No dia 4, depois de automatizar o pipeline e preparar os dados para a inteligência artificial, **ele não precisa mais de você para perguntar**.

CAPÍTULO 2

Antes do SQL: os conceitos

Tabela, linha, coluna, tipo e chave. Cinco palavras que resolvem tudo.

Se você já usou Excel, já entende 80% do que vem a seguir. Um banco de dados é parecido com uma pasta de planilhas, com duas diferenças: ele guarda muito mais linhas e **não deixa** você escrever qualquer coisa em qualquer lugar.

Tabela, linha e coluna

Termo	No Excel seria	No nosso projeto
Tabela	Uma planilha	<code>vendas</code> , <code>produtos</code> , <code>clientes</code>
Linha (ou registro)	Uma linha da planilha	Uma venda que aconteceu
Coluna (ou campo)	Uma coluna da planilha	<code>quantidade</code> , <code>preco_unitario</code>
Tipo	Formato da célula	Texto, número inteiro, decimal, data

Tipos de dado

Toda coluna tem um tipo, e o tipo define o que cabe nela. Os quatro que você vai encontrar hoje:

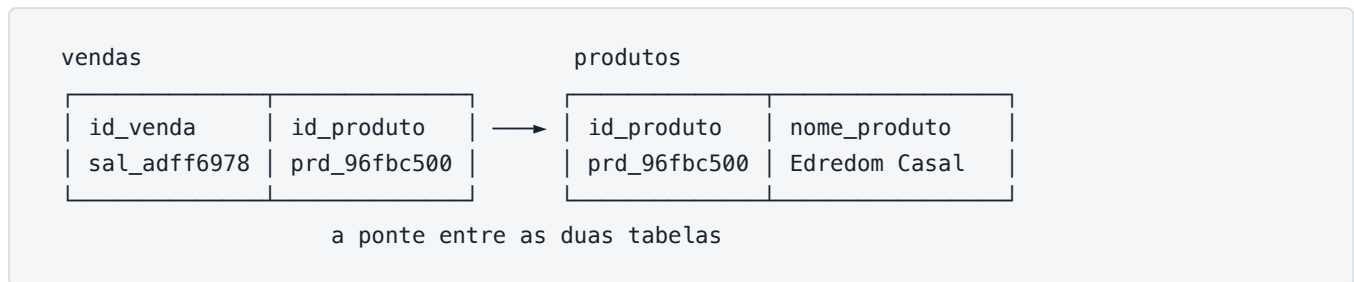
Tipo	Guarda	Exemplo no projeto
<code>STRING</code>	Texto	<code>canal_venda</code> = 'ecommerce'
<code>INT</code> / <code>BIGINT</code>	Número inteiro	<code>quantidade</code> = 2
<code>DOUBLE</code> / <code>DECIMAL</code>	Número com casas decimais	<code>preco_unitario</code> = 64.79
<code>TIMESTAMP</code>	Data e hora	<code>data_venda</code> = 13/12/2025 17:38

POR QUE O TIPO IMPORTA

Texto vai entre aspas simples na consulta (`'ecommerce'`), número não vai (`quantidade > 1`). E você só consegue somar o que é número. Metade dos erros de iniciante vem de confundir esses dois.

Chave: o crachá de cada linha

Cada venda tem um `id_venda`, cada produto um `id_produto`, cada cliente um `id_cliente`. Esse identificador único é a **chave**. É por ela que as tabelas se conversam: a tabela de vendas guarda apenas o *código* do produto, e não o nome.



Juntar as duas tabelas por essa ponte é o trabalho do **JOIN**, que você aprende no capítulo 14.

O que é uma consulta

Uma **consulta** (em inglês, *query*) é uma pergunta escrita em SQL. Você descreve o resultado que quer, e o banco decide como buscar. Ele nunca muda os seus dados quando você só consulta: dá para errar à vontade e rodar de novo.

CAPÍTULO 3

O que é SQL

A linguagem que você diz o que quer, não como fazer.

SQL (*Structured Query Language*) é a linguagem padrão para conversar com bancos de dados. Existe desde os anos 1970 e continua sendo a habilidade mais pedida em vagas de dados, porque quase tudo (Databricks, Postgres, BigQuery, Excel moderno, Power BI) fala SQL.

Declarativa: você descreve o resultado

Em linguagens como Python você escreve o *passo a passo*. Em SQL você descreve o *resultado*, e o banco escolhe o caminho mais rápido sozinho.

CONSULTA

```
-- Você diz: "quero os 10 produtos mais caros"
SELECT nome_produto, preco_atual
FROM ecommerce.bronze.produtos
ORDER BY preco_atual DESC
LIMIT 10;
```

Você não disse em que ordem ler os arquivos, nem como ordenar. Só disse o que queria ver.

SQL é quase inglês

Leia a consulta acima em voz alta, traduzindo: *selecione o nome e o preço, da tabela produtos, ordene pelo preço do maior para o menor, e me dê 10 linhas*. É literalmente isso.

Comando	Em português	Para que serve
<code>SELECT</code>	selecione	Escolher colunas
<code>FROM</code>	de	Dizer a tabela
<code>WHERE</code>	onde	Filtrar linhas
<code>GROUP BY</code>	agrupe por	Agrupar para totalizar
<code>HAVING</code>	tendo	Filtrar grupos
<code>ORDER BY</code>	ordene por	Ordenar o resultado
<code>LIMIT</code>	limite	Cortar o resultado
<code>JOIN</code>	junte	Combinar tabelas

ANTES DE DECORAR, ENTENDA A LEITURA

SQL é muito mais interpretação do que memorização. Se você consegue ler a consulta como uma frase, você consegue escrever a próxima.

CAPÍTULO 4

O ambiente: Databricks

Catálogo, schema, tabela, notebook e serverless, sem mistério.

Todo o projeto roda no **Databricks Free Edition**, que é gratuito, não pede cartão de crédito e funciona no navegador. Você não instala nada.

O endereço de cada dado

No Databricks, toda tabela tem um endereço de três partes, separado por pontos. Quem organiza isso é o **Unity Catalog**.

```
catálogo . schema . tabela
ecommerce . bronze . vendas
|           |           |
|           |           | onde os dados estão
|           |           | a camada (bronze, silver, gold)
|           |           | o projeto
```

Por isso as consultas deste guia sempre escrevem o nome completo: `ecommerce.bronze.vendas`.

As camadas: bronze, silver e gold

Camada	O que guarda	Quando aparece
bronze	O dado como chegou da origem, sem tratamento	Hoje, na Aula 1
silver	Dado limpo, com tipos certos e problemas marcados	Aula 2
gold	Tabelas prontas para o negócio, uma por pergunta	Aula 2

Hoje você trabalha só na bronze. Ela recebe os 4 arquivos CSV do projeto, exatamente como vieram.

Notebook e serverless

O **notebook** é o documento onde você escreve as consultas. Ele é dividido em **células**: você escreve uma consulta por célula e roda com `Shift + Enter`. O resultado aparece logo abaixo.

Serverless significa que você não liga nem desliga servidor nenhum. Ao rodar a primeira célula, o Databricks liga uma máquina para você em segundos e desliga sozinho quando você para.

SE TRAVAR NA AULA AO VIVO

Ambientes na nuvem às vezes demoram ou caem. Se acontecer, siga lendo este guia: todas as consultas e todos os resultados estão aqui. A gravação também fica disponível, então não se preocupe em copiar tudo na hora.

Subindo os CSVs como tabelas

Os 4 arquivos estão na pasta `dados/` do repositório. Para cada um deles:

1. Clique em **+ New** → **Add or upload data** → **Create or modify table**.
2. Arraste o arquivo CSV.
3. Escolha o catálogo `ecommerce` e o schema `bronze`.
4. Mantenha o nome sugerido para a tabela (igual ao do arquivo).
5. Confira a prévia e clique em **Create table**.

SETUP

```
-- antes do upload, crie o catálogo e os schemas
CREATE CATALOG IF NOT EXISTS ecommerce;

CREATE SCHEMA IF NOT EXISTS ecommerce.bronze;
CREATE SCHEMA IF NOT EXISTS ecommerce.silver;
CREATE SCHEMA IF NOT EXISTS ecommerce.gold;
```

Depois do upload, confira se tudo chegou:

CONSULTA

```
SELECT 'vendas' AS tabela, COUNT(*) AS linhas FROM ecommerce.bronze.vendas
UNION ALL SELECT 'produtos', COUNT(*) FROM ecommerce.bronze.produtos
UNION ALL SELECT 'clientes', COUNT(*) FROM ecommerce.bronze.clientes
UNION ALL SELECT 'preco_competidores', COUNT(*) FROM ecommerce.bronze.preco_competidores;
```

tabela	linhas
vendas	3.020
produtos	215
clientes	50
preco_competidores	728

CAPÍTULO 5

As quatro tabelas

O que tem dentro de cada uma, com dados de verdade.

Antes de escrever qualquer consulta, conheça os dados. Abaixo, as primeiras linhas de cada tabela, exatamente como elas aparecem no Databricks.

vendas · 3.020 linhas

Uma linha por venda. É a tabela principal, a que registra o que aconteceu.

RESULTADO

ID_VENDA	DATA_VENDA	CANAL	QTD	PREÇO UNITÁRIO
sal_adff6978b0c6	13/12/2025 17:38	loja_fisica	2	64,79
sal_0648f0d17798	13/12/2025 10:32	ecommerce	1	89,50
sal_2cd165121973	13/12/2025 11:55	loja_fisica	2	69,69
sal_0cdb7d67970c	13/12/2025 20:48	ecommerce	1	80,99
sal_0ed16afae5b1	13/12/2025 22:15	ecommerce	1	190,71

Coluna	O que é
<code>id_venda</code>	Identificador único da venda
<code>data_venda</code>	Data e hora em que a venda aconteceu
<code>id_cliente</code>	Código do cliente que comprou
<code>id_produto</code>	Código do produto vendido
<code>canal_venda</code>	<code>ecommerce</code> (site) ou <code>loja_fisica</code>
<code>quantidade</code>	Quantas unidades foram vendidas
<code>preco_unitario</code>	Preço de cada unidade, em reais

A CONTA MAIS IMPORTANTE DO PROJETO

A tabela **não tem** uma coluna de receita. A receita de cada venda é **quantidade × preço_unitario**. Você vai escrever isso dezenas de vezes hoje.

produtos · 215 linhas

RESULTADO

ID_PRODUTO	PRODUTO	CATEGORIA	MARCA	PREÇO ATUAL
prd_2293732b7542	Cortina Blackout	Casa	Xiaomi	68,90
prd_84a009f1889d	Cortina Box	Casa	Consul	32,90
prd_ff35b88df534	SSD 500GB	Eletrônicos	Acer	211,90
prd_38c1b898aecc	Microfone para Stream	Games	Samsung	140,90
prd_4ee65ed59110	Frigideira Antiaderente	Cozinha	Philips	41,90

clientes · 50 linhas

RESULTADO

ID_CLIENTE	CLIENTE	UF	PAÍS
cus_71308b76151a	Srta. Amanda Sousa	MS	Brasil
cus_0914bdf6e1fe	Isabelly Câmara	MT	Brasil
cus_d36c11b261a6	Cauã Rocha	MA	Brasil
cus_d2762f5968fe	Dra. Aurora Pastor	SE	Brasil
cus_a01814ad9247	Ana Beatriz Alves	SP	Brasil

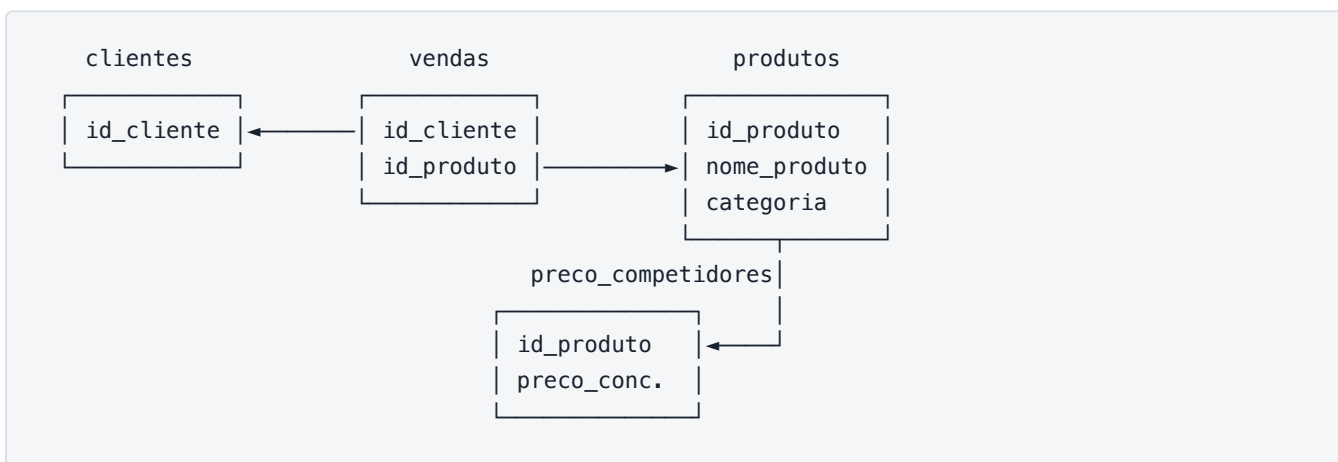
preco_competidores · 728 linhas

Uma linha por produto e concorrente. O mesmo produto aparece até 4 vezes, uma para cada loja.

RESULTADO

ID_PRODUTO	CONCORRENTE	PREÇO DO CONCORRENTE
prd_2293732b7542	Mercado Livre	65,45
prd_2293732b7542	Amazon	67,52
prd_2293732b7542	Shopee	70,97
prd_84a009f1889d	Mercado Livre	31,25
prd_84a009f1889d	Amazon	31,25

Como elas se ligam



CAPÍTULO 6

Anatomia de uma consulta

As sete cláusulas, na ordem certa de escrita e de execução.

Toda consulta SQL usa as mesmas peças, sempre nesta ordem. Só **SELECT** e **FROM** são obrigatórios; o resto entra quando a pergunta pede.

ESTRUTURA

SELECT	colunas e contas	-- o que mostrar
FROM	tabela	-- de onde
JOIN	outra tabela ON ponte	-- juntar tabelas (opcional)
WHERE	filtro de linhas	-- antes de agrupar (opcional)
GROUP BY	agrupamento	-- uma linha por grupo (opcional)
HAVING	filtro de grupos	-- depois de agrupar (opcional)
ORDER BY	ordenação	-- ordenar (opcional)
LIMIT	quantas linhas	-- cortar (opcional)

TROCAR A ORDEM DÁ ERRO

Escrever `WHERE` antes do `FROM`, ou `ORDER BY` antes do `GROUP BY`, resulta em erro de sintaxe. A ordem acima é fixa.

A ordem em que o banco executa

Você escreve em uma ordem, mas o banco executa em outra. Entender isso resolve metade dos erros de iniciante, principalmente a confusão entre `WHERE` e `HAVING`.

Escrita	Execução	O que acontece
1. <code>SELECT</code>	5º	Calcula e escolhe as colunas
2. <code>FROM / JOIN</code>	1º	Lê e junta as tabelas
3. <code>WHERE</code>	2º	Filtra linhas
4. <code>GROUP BY</code>	3º	Forma os grupos
5. <code>HAVING</code>	4º	Filtra grupos
6. <code>ORDER BY</code>	6º	Ordena o resultado
7. <code>LIMIT</code>	7º	Corta o resultado

POR QUE ISSO EXPLICA O ERRO MAIS COMUM

Quando o `WHERE` roda, os grupos ainda não existem. Por isso não dá para escrever `WHERE SUM(...) > 1000`. Para filtrar por um total, use `HAVING`, que roda depois do agrupamento (capítulo 13).

Comentários e ponto e vírgula

Duas conveniências: tudo depois de `--` é comentário e o banco ignora; o ponto e vírgula marca o fim da consulta e é obrigatório quando você escreve várias na mesma célula.

CAPÍTULO 7

SELECT e FROM

A primeira pergunta: o que tem nesta tabela?

`SELECT` diz **quais colunas** você quer ver e `FROM` diz **de qual tabela**. O asterisco `*` significa “todas as colunas”.

CONSULTA

```
SELECT *
FROM ecommerce.bronze.vendas;
```

RESULTADO

ID_VENDA	DATA_VENDA	ID_CLIENTE	ID_PRODUTO	CANAL	QTD	PREÇO
sal_adff6978b0c6	13/12/2025 17:38	cus_508a15ccf0fa	prd_96fbc500aec1	loja_fisica	2	64,79
sal_0648f0d17798	13/12/2025 10:32	cus_f64907d41a69	prd_686e2aab873d	ecommerce	1	89,50
sal_2cd165121973	13/12/2025 11:55	cus_78ab1cf16e12	prd_223c9b6a0a76	loja_fisica	2	69,69
sal_0cdb7d67970c	13/12/2025 20:48	cus_2b1b3e2a1515	prd_30518d908a92	ecommerce	1	80,99
sal_0ed16afae5b1	13/12/2025 22:15	cus_644bf6820c61	prd_ff35b88df534	ecommerce	1	190,71

O Databricks mostra as primeiras linhas na tela e avisa quantas existem no total. Aqui são 3.020.

Escolhendo colunas

Na prática quase nunca queremos todas as colunas. Escreva só as que interessam, separadas por vírgula. O resultado fica mais limpo e a consulta mais rápida.

CONSULTA

```
SELECT
  id_venda,
  canal_venda,
  quantidade,
  preco_unitario
FROM ecommerce.bronze.vendas;
```

RESULTADO

ID_VENDA	CANAL	QTD	PREÇO UNITÁRIO
sal_adff6978b0c6	loja_fisica	2	64,79
sal_0648f0d17798	ecommerce	1	89,50
sal_2cd165121973	loja_fisica	2	69,69
sal_0cdb7d67970c	ecommerce	1	80,99
sal_0ed16afae5b1	ecommerce	1	190,71

ESQUECEU A VÍRGULA

`SELECT id_venda canal_venda` não dá erro, mas devolve uma coluna só, chamada `canal_venda`: sem a vírgula, o SQL entende que você apelidou a coluna. Se sumiu uma coluna do resultado, procure a vírgula.

SUA VEZ

Escreva uma consulta que mostre `nome_produto`, `categoria` e `preco_atual` da tabela `ecommerce.bronze.produtos`.

CAPÍTULO 8

Colunas calculadas e AS

Criando a receita, que não existe na tabela.

O `SELECT` não serve só para mostrar colunas: ele também **calcula**. Você pode somar, subtrair, multiplicar e dividir colunas, e o resultado vira uma coluna nova.

Operador	Faz	Exemplo
+	Soma	<code>preco_atual + 10</code>
-	Subtrai	<code>preco_atual - custo</code>
*	Multiplica	<code>quantidade * preco_unitario</code>
/	Divide	<code>receita / 2</code>

Dando nome ao resultado: AS

Sem apelido, a coluna nova aparece com o nome da própria conta. O `AS` resolve isso.

CONSULTA

```
SELECT
  id_venda,
  quantidade,
  preco_unitario,
  quantidade * preco_unitario AS receita
FROM ecommerce.bronze.vendas;
```

RESULTADO

ID_VENDA	QTD	PREÇO UNITÁRIO	RECEITA
sal_e256b384597c	5	1.253,99	6.269,95
sal_9cab7a0cb204	5	1.149,90	5.749,50
sal_b46ba216cf38	4	1.286,09	5.144,36
sal_6ab945691dab	4	1.214,64	4.858,56
sal_6e559fabe4dc	4	1.192,41	4.769,64

NOMES COM ACENTO OU ESPAÇO

Prefira nomes simples: `receita`, `ticket_medio`. Se precisar de espaço ou acento, use crase: `AS `Receita Total``.

Arredondando com ROUND

Contas com decimais costumam devolver muitas casas. `ROUND(valor, 2)` deixa em duas casas, como em reais.

CONSULTA

```
SELECT
  id_venda,
  ROUND(quantidade * preco_unitario, 2) AS receita
FROM ecommerce.bronze.vendas;
```

SUA VEZ

Crie uma coluna `preco_com_desconto` na tabela de produtos, com 10% de desconto (`preco_atual * 0.9`), arredondada em duas casas.

CAPÍTULO 9

ORDER BY e LIMIT

Ordenar o resultado e ficar só com o topo.

ORDER BY: colocando em ordem

`ORDER BY` ordena o resultado por uma ou mais colunas. É o “classificar” do Excel.

Palavra	Ordem	Quando usar
ASC	Crescente, do menor para o maior	Padrão, pode omitir
DESC	Decrescente, do maior para o menor	Rankings, “os maiores”

Repare que dá para ordenar por uma coluna que você mesmo criou com `AS`:

CONSULTA

```
SELECT
  id_venda,
  canal_venda,
  quantidade,
  preco_unitario,
  quantidade * preco_unitario AS receita
FROM ecommerce.bronze.vendas
ORDER BY receita DESC;
```

RESULTADO

ID_VENDA	QTD	PREÇO UNITÁRIO	RECEITA
sal_e256b384597c	5	1.253,99	6.269,95
sal_9cab7a0cb204	5	1.149,90	5.749,50
sal_b46ba216cf38	4	1.286,09	5.144,36
sal_6ab945691dab	4	1.214,64	4.858,56
sal_6e559fabe4dc	4	1.192,41	4.769,64

Para desempatar, use mais de uma coluna: `ORDER BY categoria, preco_atual DESC` ordena por categoria e, dentro de cada categoria, do mais caro para o mais barato.

LIMIT: cortando o resultado

A tabela tem 3.020 linhas e ninguém lê 3.020 linhas. `LIMIT` corta. Junto com `ORDER BY`, ele forma o padrão mais usado em dashboards: o **top N**.

CONSULTA

```
SELECT
  id_venda,
  quantidade * preco_unitario AS receita
FROM ecommerce.bronze.vendas
ORDER BY receita DESC
LIMIT 10;
```

LIMIT SEM ORDER BY

`LIMIT 10` sozinho devolve 10 linhas **quaisquer**, e elas podem mudar a cada execução. Se você quer “os 10 maiores”, o `ORDER BY` é obrigatório.

SUA VEZ

Quais são os 5 produtos mais caros do catálogo? E os 5 mais baratos?

WHERE: filtrando linhas

Só me mostre o que interessa.

WHERE filtra **linhas**: só passam as que atendem à condição. Ele vem sempre depois do **FROM** e antes do **ORDER BY**.

CONSULTA

```
SELECT
  id_venda,
  canal_venda,
  quantidade,
  preco_unitario
FROM ecommerce.bronze.vendas
WHERE canal_venda = 'loja_fisica'
AND quantidade > 1;
```

RESULTADO

ID_VENDA	CANAL	QTD	PREÇO UNITÁRIO
sal_adff6978b0c6	loja_fisica	2	64,79
sal_2cd165121973	loja_fisica	2	69,69
sal_08d113b117c7	loja_fisica	2	438,21
sal_5457926e6ab9	loja_fisica	2	109,90
sal_4368a147d186	loja_fisica	2	1.286,09

Os operadores de comparação

Operador	Significa	Exemplo
=	Igual a	canal_venda = 'ecommerce'
<> ou !=	Diferente de	canal_venda <> 'ecommerce'
> <	Maior / menor	preco_unitario > 500
>= <=	Maior ou igual / menor ou igual	quantidade >= 2
BETWEEN	Entre dois valores (inclusive)	preco_atual BETWEEN 100 AND 500
IN	Está na lista	categoria IN ('Moda', 'Games')
LIKE	Parece com (% = qualquer coisa)	nome_produto LIKE 'Tênis%'
IS NULL	Está vazio	id_produto IS NULL

TEXTO ENTRE ASPAS, NÚMERO SEM ASPAS

`WHERE canal_venda = 'ecommerce'` (texto, com aspas simples) e `WHERE quantidade > 1` (número, sem aspas). Aspas duplas não valem para texto em SQL.

Combinando condições

Conector	Regra	Exemplo
AND	As duas condições precisam ser verdadeiras	e-commerce e acima de R\$ 500
OR	Basta uma ser verdadeira	Moda <i>ou</i> Games
NOT	Inverte a condição	<code>NOT canal_venda = 'ecommerce'</code>

CONSULTA

```
SELECT nome_produto, categoria, preco_atual
FROM ecommerce.bronze.produtos
WHERE categoria IN ('Eletrônicos', 'Games')
AND preco_atual BETWEEN 100 AND 500
ORDER BY preco_atual DESC;
```

MISTURAR AND E OR SEM PARÊNTESES

`WHERE a = 1 OR b = 2 AND c = 3` não é o que parece: o `AND` é avaliado primeiro. Sempre use parênteses quando misturar os dois: `WHERE (a = 1 OR b = 2) AND c = 3`.

DISTINCT: quais valores existem?

Antes de filtrar por uma coluna, vale saber o que existe nela. `DISTINCT` remove as repetições.

CONSULTA

```
SELECT DISTINCT canal_venda
FROM ecommerce.bronze.vendas;
```

RESULTADO

CANAL_VENDA

loja_fisica

ecommerce

SUA VEZ

Quantas categorias diferentes existem no catálogo? (Dica: `SELECT DISTINCT categoria`.) E quais vendas foram feitas no e-commerce com mais de 3 unidades?

CAPÍTULO 11

Agregações: de 3.020 linhas para um número

COUNT, SUM, AVG, MIN e MAX. A primeira resposta ao Diretor Comercial.

Até aqui todo resultado tinha uma linha por venda. O diretor não quer 3.020 linhas: ele quer **um número**. As funções de agregação resumem muitas linhas em uma só.

Função	O que faz	Exemplo
<code>COUNT(*)</code>	Conta linhas	Quantas vendas houve
<code>COUNT(DISTINCT col)</code>	Conta valores diferentes	Quantos clientes compraram
<code>SUM(col)</code>	Soma	Receita total
<code>AVG(col)</code>	Média	Ticket médio
<code>MIN(col) / MAX(col)</code>	Menor e maior valor	Primeira e última venda

Uma de cada vez

Comece simples. Quantas vendas temos?

CONSULTA

```
SELECT COUNT(*) AS total_vendas
FROM ecommerce.bronze.vendas;
```

Agora a receita total. Repare que a conta vai **dentro** do `SUM`:

CONSULTA

```
SELECT SUM(quantidade * preco_unitario) AS receita_total
FROM ecommerce.bronze.vendas;
```

O painel de indicadores

Uma consulta só, com todos os números que o Diretor Comercial pediu:

CONSULTA

```
SELECT
  COUNT(*) AS total_vendas,
  ROUND(SUM(quantidade * preco_unitario), 2) AS receita_total,
  ROUND(AVG(quantidade * preco_unitario), 2) AS ticket_medio,
  COUNT(DISTINCT id_cliente) AS clientes_unicos
FROM ecommerce.bronze.vendas;
```

RESULTADO

TOTAL DE VENDAS	RECEITA TOTAL	TICKET MÉDIO	CLIENTES ÚNICOS
3.020	974.077,28	322,54	50

RESPOSTA AO DIRETOR COMERCIAL

R\$ 974.077,28 em 3.020 vendas, com ticket médio de **R\$ 322,54**, de 50 clientes diferentes.

COUNT(*) × COUNT(DISTINCT)

COUNT(*) conta linhas: 3.020 vendas. COUNT(DISTINCT id_cliente) conta valores diferentes: 50 clientes. Um cliente que comprou 60 vezes aparece uma vez só.

MÉDIA DE MÉDIA NÃO EXISTE

O ticket médio é `AVG(quantidade * preco_unitario)`, ou então `SUM(receita) / COUNT(*)`. Calcular a média de uma coluna que já é média devolve um número errado, e esse é um dos erros mais comuns em relatórios reais.

SUA VEZ

Qual o preço médio, o mais caro e o mais barato do catálogo de produtos?

CAPÍTULO 12

GROUP BY: o mesmo cálculo por grupo

Receita por canal, por dia, por categoria. A pergunta “e por...?”.

Quando o diretor ouve “vendemos R\$ 974 mil”, a próxima pergunta é sempre a mesma: “e quanto por canal?”. O `GROUP BY` junta as linhas em grupos e aplica a agregação **dentro de cada grupo**.

sem GROUP BY

receita total
974.077,28

1 linha

com GROUP BY canal_venda

ecommerce	705.486,21
loja_fisica	268.591,07

1 linha por canal

Passo a passo

Primeiro só contando as vendas de cada canal:

CONSULTA

```
SELECT
  canal_venda,
  COUNT(*) AS total_vendas
FROM ecommerce.bronze.vendas
GROUP BY canal_venda;
```

Agora acrescente receita e ticket médio, e ordene pela receita:

CONSULTA

```
SELECT
  canal_venda,
  COUNT(*) AS total_vendas,
  ROUND(SUM(quantidade * preco_unitario), 2) AS receita_total,
  ROUND(AVG(quantidade * preco_unitario), 2) AS ticket_medio
FROM ecommerce.bronze.vendas
GROUP BY canal_venda
ORDER BY receita_total DESC;
```

RESULTADO

CANAL	VENDAS	RECEITA TOTAL	TICKET MÉDIO
ecommerce	2.155	705.486,21	327,37
loja_fisica	865	268.591,07	310,51

RESPOSTA AO DIRETOR COMERCIAL

O **e-commerce** vende 2,5 vezes mais que a loja física em número de vendas (2.155 contra 865) e responde por **72% da receita**. O ticket médio dos dois canais é parecido.

A REGRA DE OURO DO GROUP BY

Toda coluna do `SELECT` precisa estar no `GROUP BY` ou dentro de uma agregação. Se faltar, o Databricks responde `MISSING_AGGREGATION`: ele está perguntando “você pediu essa coluna, mas qual valor eu mostro para o grupo inteiro?”.

Agrupando por data

`DATE(data_venda)` corta o horário e deixa só o dia. Assim, todas as vendas de um mesmo dia caem no mesmo grupo.

CONSULTA

```
SELECT
  DATE(data_venda)           AS dia,
  COUNT(*)                   AS total_vendas,
  ROUND(SUM(quantidade * preco_unitario), 2) AS receita_total
FROM ecommerce.bronze.vendas
GROUP BY DATE(data_venda)
ORDER BY dia;
```

RESULTADO

DIA	VENDAS	RECEITA
13/12/2025	100	27.980,74
14/12/2025	102	40.740,48
15/12/2025	100	36.409,09
16/12/2025	101	30.794,21
17/12/2025	101	38.109,98

VENDO COMO GRÁFICO

Depois de rodar, clique em `+` → **Visualization** no resultado e escolha **Line**. Em dois cliques você tem a série de receita diária, que vai virar um dos gráficos do dashboard.

Outras funções de data úteis

Função	Devolve	Exemplo de uso
<code>DATE(coluna)</code>	Só a data	Agrupar por dia
<code>YEAR(coluna) / MONTH(coluna)</code>	Ano / mês como número	Agrupar por mês
<code>HOUR(coluna)</code>	Hora (0 a 23)	Descobrir o horário de pico
<code>DATE_TRUNC('WEEK', coluna)</code>	Início da semana	Agrupar por semana
<code>DATE_FORMAT(coluna, 'EEEE')</code>	Dia da semana por extenso	Comparar dias da semana

SUA VEZ

Qual o horário do dia com mais vendas? (Dica: `GROUP BY HOUR(data_venda)`.)

CAPÍTULO 13

HAVING: filtrando grupos

A diferença entre WHERE e HAVING, de uma vez por todas.

Os dois filtram. A diferença é **quando**.

	WHERE	HAVING
Filtra	Linhas, uma a uma	Grupos, depois de somados
Roda	Antes do <code>GROUP BY</code>	Depois do <code>GROUP BY</code>
Aceita agregação	Não	Sim
Exemplo	<code>WHERE canal_venda = 'ecommerce'</code>	<code>HAVING SUM(receita) > 50000</code>

CONSULTA

```
SELECT
  DATE(data_venda) AS dia,
  ROUND(SUM(quantidade * preco_unitario), 2) AS receita_total
FROM ecommerce.bronze.vendas
WHERE canal_venda = 'ecommerce' -- filtra vendas, antes de agrupar
GROUP BY DATE(data_venda)
HAVING SUM(quantidade * preco_unitario) > 25000 -- filtra dias, depois de agrupar
ORDER BY receita_total DESC;
```

Leia a consulta como uma frase: *considerando só as vendas do e-commerce, agrupe por dia e me mostre apenas os dias que passaram de R\$ 25 mil.*

AGREGAÇÃO NO WHERE

Escrever `WHERE SUM(receita) > 25000` devolve um erro parecido com `AGGREGATE_FUNCTION_WITH_NONDETERMINISTIC_EXPRESSION` ou `INVALID_WHERE_CONDITION`. Faz sentido: no momento do `WHERE`, a soma ainda não foi calculada. Troque por `HAVING`.

SUA VEZ

Quais categorias de produto têm mais de 20 produtos cadastrados? (Dica: `GROUP BY categoria HAVING COUNT(*) > 20`.)

CAPÍTULO 14

JOIN: juntando tabelas

O comando que transforma códigos em nomes.

A tabela `vendas` guarda só o **código** do produto. Nome e categoria estão em `produtos`. Sem juntar as duas, não dá para responder “qual categoria vende mais”.

Primeiro, na mão

Esta é a primeira venda da tabela:

CONSULTA

```
SELECT id_venda, id_produto, quantidade, preco_unitario
FROM ecommerce.bronze.vendas
WHERE id_venda = 'sal_adff6978b0c6';
```

Ela vendeu o produto `prd_96fbc500aec1`. Que produto é esse? Procurando na outra tabela:

CONSULTA

```
SELECT id_produto, nome_produto, categoria, marca
FROM ecommerce.bronze.produtos
WHERE id_produto = 'prd_96fbc500aec1';
```

Um **Edredom Casal**, da categoria Casa. Fazer isso na mão 3.020 vezes é impossível: é para isso que existe o `JOIN`.

Agora com JOIN

vendas

id_venda	id_produto
sal_adff...	prd_96fb...

ON

⇒

produtos

id_produto	nome_produto	categoria
prd_96fb...	Edredom Casal	Casa

v.id_produto = p.id_produto

A consulta tem três novidades:

- `JOIN ecommerce.bronze.produtos p`: a tabela que entra, com o apelido `p`.
- `ON v.id_produto = p.id_produto`: **a ponte**, a coluna em comum.
- `v.` e `p.` antes de cada coluna: de qual tabela ela vem.

CONSULTA

```
SELECT
  v.id_venda,
  v.id_produto,
  p.nome_produto,
  p.categoria
FROM ecommerce.bronze.vendas v
JOIN ecommerce.bronze.produtos p
  ON v.id_produto = p.id_produto;
```

RESULTADO

ID_VENDA	ID_PRODUTO	PRODUTO	CATEGORIA
sal_adff6978b0c6	prd_96fbc500aec1	Edredom Casal	Casa
sal_0648f0d17798	prd_686e2aab873d	Blusa de Frio	Moda
sal_2cd165121973	prd_223c9b6a0a76	Luminária de Mesa	Casa
sal_0cdb7d67970c	prd_30518d908a92	Cinto de Couro	Acessórios
sal_0ed16afae5b1	prd_ff35b88df534	SSD 500GB	Eletrônicos

APELIDOS (ALIAS)

`FROM ecommerce.bronze.vendas v` apelida a tabela de `v`. A partir daí você escreve `v.quantidade` em vez do nome completo. Com duas ou mais tabelas, o apelido deixa de ser conveniência e vira necessidade, porque as duas têm a coluna `id_produto`.

JOIN + GROUP BY: a resposta do diretor

CONSULTA

```
SELECT
  p.categoria,
  COUNT(*) AS total_vendas,
  ROUND(SUM(v.quantidade * v.preco_unitario), 2) AS receita_total
FROM ecommerce.bronze.vendas v
JOIN ecommerce.bronze.produtos p
  ON v.id_produto = p.id_produto
GROUP BY p.categoria
ORDER BY receita_total DESC;
```

RESULTADO

CATEGORIA	VENDAS	RECEITA TOTAL
Moda	508	248.124,15
Áudio	194	137.061,47
Acessórios	312	120.909,94
Casa	575	112.704,52
Cozinha	275	85.059,89
Beleza	197	66.852,52

Os 10 produtos com mais receita

CONSULTA

```
SELECT
  p.nome_produto,
  p.categoria,
  SUM(v.quantidade) AS itens_vendidos,
  ROUND(SUM(v.quantidade * v.preco_unitario), 2) AS receita_total
FROM ecommerce.bronze.vendas v
JOIN ecommerce.bronze.produtos p
  ON v.id_produto = p.id_produto
GROUP BY p.nome_produto, p.categoria
ORDER BY receita_total DESC
LIMIT 10;
```

RESULTADO

PRODUTO	CATEGORIA	ITENS	RECEITA
Fone de Ouvido Esportivo	Áudio	106	120.457,54
Camisa Social	Moda	103	115.794,92
Necessaire	Acessórios	57	63.668,43
Persiana Vertical	Casa	192	46.897,53
Calça Jeans Skinny	Moda	88	41.313,34
Condicionador Hidratante	Beleza	71	31.587,75
Shorts Jeans	Moda	76	28.431,49
Garrafa Térmica 1L	Cozinha	81	24.221,94
Vestido Floral	Moda	125	24.132,62
Notebook Inspiron 15	Eletrônicos	111	22.107,07

INNER JOIN × LEFT JOIN: a pegadinha do dia

Sempre confira os totais depois de um `JOIN`. Compare a receita com e sem ele:

CONSULTA

```
SELECT 'sem JOIN' AS consulta, ROUND(SUM(quantidade * preco_unitario), 2) AS receita
FROM ecommerce.bronze.vendas
UNION ALL
SELECT 'com JOIN', ROUND(SUM(v.quantidade * v.preco_unitario), 2)
FROM ecommerce.bronze.vendas v
JOIN ecommerce.bronze.produtos p ON v.id_produto = p.id_produto;
```

consulta	receita
sem JOIN	974.077,28
com JOIN	969.837,27

Sumiram **R\$ 4.240,01**. O motivo: o `JOIN` que usamos é um `INNER JOIN`, que só mantém as linhas que encontram par nas duas tabelas. Vendas de produtos que não estão no catálogo são descartadas **em silêncio**.

Tipo	Mantém	Use quando
<code>JOIN (INNER JOIN)</code>	Só o que casa nas duas tabelas	Você quer apenas o que tem par
<code>LEFT JOIN</code>	Tudo da tabela da esquerda, com <code>NULL</code> onde não casou	Você não pode perder nada da tabela principal

Com `LEFT JOIN` e um filtro `IS NULL`, achamos exatamente as vendas órfãs:

CONSULTA

```
SELECT
  v.id_venda,
  v.id_produto,
  p.nome_produto,
  ROUND(v.quantidade * v.preco_unitario, 2) AS receita
FROM ecommerce.bronze.vendas v
LEFT JOIN ecommerce.bronze.produtos p
  ON v.id_produto = p.id_produto
WHERE p.id_produto IS NULL;
```

RESULTADO

ID_VENDA	ID_PRODUTO	RECEITA
sal_6265f7732b1b	prd_2b1b2e2c60e1	85,90
sal_c3cf31c81651	prd_c366b8673062	223,19
sal_27c5e374a2db	prd_b3d5d6aac3ff	26,99
sal_b1ad3fa0f173	prd_7f61cfa619e3	87,90
sal_e0cec57ebe4d	prd_235a85c800bf	278,98

20 VENDAS SEM PRODUTO CADASTRADO

Isso é um problema de **qualidade de dados**, não de SQL. Ele volta na Aula 2, quando a camada silver passa a marcar essas vendas, e na Aula 3, quando um teste automático avisa se o problema crescer. Por enquanto, fica a regra: **sempre confira os totais depois de um JOIN**.

SUA VEZ

Quais produtos **nunca** foram vendidos? (Dica: `LEFT JOIN` começando pela tabela `produtos` e filtrando `v.id_venda IS NULL`. São 30.)

CAPÍTULO 15

Respondendo a Diretoria de Clientes

Os 10 melhores clientes e a distribuição por estado.

A pergunta é: “quem são nossos 10 melhores clientes e de quais estados eles são?”. Vamos em três passos, sem pressa.

Passo 1: receita por cliente

Só com a tabela de vendas, sem `JOIN`:

CONSULTA

```
SELECT
  id_cliente,
  ROUND(SUM(quantidade * preco_unitario), 2) AS receita_total
FROM ecommerce.bronze.vendas
GROUP BY id_cliente
ORDER BY receita_total DESC
LIMIT 10;
```

Funciona, mas a diretora não conhece o cliente `cus_2b1b3e2a1515`. Ela quer o nome.

Passo 2: trazendo o nome

CONSULTA

```
SELECT
  c.nome_cliente,
  ROUND(SUM(v.quantidade * v.preco_unitario), 2) AS receita_total
FROM ecommerce.bronze.vendas v
JOIN ecommerce.bronze.clientes c
  ON v.id_cliente = c.id_cliente
GROUP BY c.nome_cliente
ORDER BY receita_total DESC
LIMIT 10;
```

Passo 3: a consulta completa

CONSULTA

```
SELECT
  c.nome_cliente,
  c.estado,
  COUNT(*) AS total_compras,
  ROUND(SUM(v.quantidade * v.preco_unitario), 2) AS receita_total,
  ROUND(AVG(v.quantidade * v.preco_unitario), 2) AS ticket_medio
FROM ecommerce.bronze.vendas v
JOIN ecommerce.bronze.clientes c
  ON v.id_cliente = c.id_cliente
GROUP BY c.nome_cliente, c.estado
ORDER BY receita_total DESC
LIMIT 10;
```

RESULTADO

CLIENTE	UF	COMPRAS	RECEITA	TICKET MÉDIO
Ana Sophia Pereira	MG	53	30.716,63	579,56
Melissa Pastor	AC	68	30.211,93	444,29
Murilo da Mata	RR	81	28.285,82	349,21
Dr. Benício Gomes	AL	72	27.424,23	380,89
Henrique da Conceição	DF	77	26.687,99	346,60
Dra. Mariane Pacheco	MT	73	25.454,83	348,70
Josué Souza	RJ	50	24.750,17	495,00
Srta. Yasmin Siqueira	PA	53	23.674,10	446,68
Bella Borges	AM	57	22.865,99	401,16
Vitor Gabriel da Luz	RS	65	22.734,53	349,76

Receita por estado

Para planejar a equipe regional, a diretora quer o mapa. É a mesma consulta, agrupando pelo estado:

CONSULTA

```
SELECT
  c.estado,
  COUNT(DISTINCT c.id_cliente) AS clientes,
  ROUND(SUM(v.quantidade * v.preco_unitario), 2) AS receita_total
FROM ecommerce.bronze.vendas v
JOIN ecommerce.bronze.clientes c
  ON v.id_cliente = c.id_cliente
GROUP BY c.estado
ORDER BY receita_total DESC;
```

RESULTADO

UF	CLIENTES	RECEITA
AM	4	79.474,67
TO	4	71.055,63
PA	4	70.003,51
AC	3	69.833,78
SP	3	60.898,63
RS	3	60.727,10
GO	3	57.898,59
MT	3	57.431,13

RESPOSTA À DIRETORIA DE CLIENTES

A melhor cliente é **Ana Sophia Pereira (MG)**, com R\$ 30.716,63 em 53 compras. Os estados com mais receita são **AM** e **TO**, e não os do Sudeste: um resultado que contraria a intuição e muda o plano da equipe regional.

CAPÍTULO 16

Respondendo a Diretoria de Pricing

Nosso preço contra o dos concorrentes.

A tabela `preco_competidores` tem uma linha por **produto** x **concorrente**. Olhe um produto só, o Tênis Puma Speedcat:

CONSULTA

```
SELECT
  p.nome_produto,
  p.preco_atual AS nosso_preco,
  pc.nome_concorrente,
  pc.preco_concorrente
FROM ecommerce.bronze.produtos p
JOIN ecommerce.bronze.preco_competidores pc
  ON p.id_produto = pc.id_produto
WHERE p.id_produto = 'prd_1317c4a1f775';
```

RESULTADO

PRODUTO	NOSSO PREÇO	CONCORRENTE	PREÇO DO CONCORRENTE
Tênis Puma Speedcat	451,90	Mercado Livre	225,95
Tênis Puma Speedcat	451,90	Amazon	225,95
Tênis Puma Speedcat	451,90	Magalu	225,95
Tênis Puma Speedcat	451,90	Shopee	225,95

Cobramos **R\$ 451,90** por um produto que os quatro concorrentes vendem a **R\$ 225,95**. O dobro.

Uma linha por produto

Agora para o catálogo inteiro. É um **GROUP BY** por produto, com três agregações sobre o preço dos concorrentes, mais a diferença percentual:

CONSULTA

```
SELECT
  p.nome_produto,
  p.categoria,
  p.preco_atual AS nosso_preco,
  ROUND(AVG(pc.preco_concorrente), 2) AS media_concorrentes,
  ROUND((p.preco_atual - AVG(pc.preco_concorrente))
    / AVG(pc.preco_concorrente) * 100, 1) AS dif_pct
FROM ecommerce.bronze.produtos p
JOIN ecommerce.bronze.preco_competidores pc
  ON p.id_produto = pc.id_produto
GROUP BY p.nome_produto, p.categoria, p.preco_atual
ORDER BY dif_pct DESC;
```

RESULTADO

PRODUTO	CATEGORIA	NOSSO PREÇO	MÉDIA DO MERCADO	DIFERENÇA %
Tênis Nike Blazer	Tênis	37,90	18,95	100,00
Tênis Nike Revolution	Tênis	174,90	87,45	100,00
Tênis Puma Speedcat	Tênis	451,90	225,95	100,00
Tênis Nike Court	Tênis	31,90	15,95	100,00
Tênis Adidas Stan Smith	Tênis	197,90	98,95	100,00
Tênis Puma Future	Tênis	67,90	33,95	100,00
Tênis Adidas Samba	Tênis	43,90	21,95	100,00
Tênis Puma Future	Tênis	123,99	61,99	100,00

Positivo quer dizer que **estamos mais caros**. Repare que o topo da lista é quase todo de tênis.

Confirmando por categoria

CONSULTA

```
SELECT
  p.categoria,
  COUNT(DISTINCT p.id_produto) AS produtos,
  ROUND(AVG((p.preco_atual - pc.preco_concorrente)
            / pc.preco_concorrente * 100), 1) AS dif_pct_media
FROM ecommerce.bronze.produtos p
JOIN ecommerce.bronze.preco_competidores pc
  ON p.id_produto = pc.id_produto
GROUP BY p.categoria
ORDER BY dif_pct_media DESC;
```

RESULTADO

CATEGORIA	PRODUTOS	DIFERENÇA % MÉDIA
Tênis	15	100,00
Beleza	12	1,40
Informática	21	1,10
Moda	23	0,80
Games	19	0,70
Acessórios	23	0,60

RESPOSTA AO DIRETOR DE PRICING

A categoria **Tênis** está **100% acima** do mercado, ou seja, cobramos o dobro. Todas as outras categorias estão praticamente no preço da concorrência. O problema tem nome e endereço.

Os produtos mais caros que TODOS os concorrentes

Aqui o **HAVING** ganha sentido: a condição compara o nosso preço com o **maior** preço do mercado, e **MAX** é uma agregação.

CONSULTA

```

SELECT
  p.nome_produto,
  p.categoria,
  p.preco_atual      AS nosso_preco,
  MAX(pc.preco_concorrente) AS maior_preco_concorrente
FROM ecommerce.bronze.produtos p
JOIN ecommerce.bronze.preco_competidores pc
  ON p.id_produto = pc.id_produto
GROUP BY p.nome_produto, p.categoria, p.preco_atual
HAVING p.preco_atual > MAX(pc.preco_concorrente)
ORDER BY p.categoria;

```

RESULTADO

PRODUTO	CATEGORIA	NOSSO PREÇO	MAIOR PREÇO DO CONCORRENTE
Porta-moedas	Acessórios	100,50	98,49
Carteira de Couro	Acessórios	66,99	65,65
Paleta de Sombras	Beleza	1.420,99	1.392,57
Máscara Facial	Beleza	111,90	109,66
Edredom Casal	Casa	425,90	417,38
Porta-retrato	Casa	84,90	83,20
Quadro Decorativo	Casa	93,99	92,11
Frigideira Antiaderente	Cozinha	41,90	39,80

São **35 produtos** no total, a maioria da categoria Tênis. Essa é a lista de alerta que o diretor leva para a reunião de precificação.

SUA VEZ

Troque o **HAVING** por **WHERE** e rode. Leia a mensagem de erro com calma: agora ela faz sentido.

CAPÍTULO 17

Bônus: CASE WHEN

O “se, então, senão” do SQL.

CASE WHEN cria uma coluna nova a partir de uma regra. É como o **SE()** do Excel, só que dá para encadear quantas condições você quiser.

ESTRUTURA

```

CASE
  WHEN condição1 THEN 'resultado 1'
  WHEN condição2 THEN 'resultado 2'
  ELSE 'resultado padrão'
END AS nome_da_coluna

```

As condições são testadas **de cima para baixo**, e a primeira que for verdadeira vence. Por isso a ordem importa: comece pela faixa mais alta.

Segmentando clientes

A Diretora de Customer Success quer separar os clientes por valor. A regra combinada foi: VIP a partir de R\$ 22 mil, TOP_TIER a partir de R\$ 17 mil, e o resto REGULAR.

CONSULTA

```

SELECT
  c.nome_cliente,
  ROUND(SUM(v.quantidade * v.preco_unitario), 2) AS receita,
  CASE
    WHEN SUM(v.quantidade * v.preco_unitario) >= 22000 THEN 'VIP'
    WHEN SUM(v.quantidade * v.preco_unitario) >= 17000 THEN 'TOP_TIER'
    ELSE 'REGULAR'
  END AS segmento
FROM ecommerce.bronze.vendas v
JOIN ecommerce.bronze.clientes c
  ON v.id_cliente = c.id_cliente
GROUP BY c.nome_cliente
ORDER BY receita DESC;

```

RESULTADO

CLIENTE	RECEITA	SEGMENTO
Ana Sophia Pereira	30.716,63	VIP
Melissa Pastor	30.211,93	VIP
Murilo da Mata	28.285,82	VIP
Dr. Benício Gomes	27.424,23	VIP
Henrique da Conceição	26.687,99	VIP
Dra. Mariane Pacheco	25.454,83	VIP

REGRA DE NEGÓCIO SE VALIDA COM DADO

A primeira versão desse projeto usava R\$ 10 mil para VIP. Com esse limite, 49 dos 50 clientes viravam VIP, e um segmento que contém todo mundo não serve para nada. Os limites atuais vieram da distribuição real: cerca de 20% dos clientes são VIP.

Bônus: window functions

Ranking e comparação sem perder o detalhe.

GROUP BY **junta** linhas: você ganha o total, mas perde o detalhe. Uma **window function** mantém todas as linhas e ainda assim calcula algo olhando para as outras.

ESTRUTURA

```
FUNCAO(coluna) OVER (PARTITION BY grupo ORDER BY ordem)
```

Parte	O que faz
OVER ()	Define a “janela”, o conjunto de linhas que a função enxerga
PARTITION BY	Divide em grupos; o cálculo recomeça em cada um
ORDER BY	Define a ordem dentro da janela (essencial para ranking e LAG)

ROW_NUMBER: ranking por categoria

CONSULTA

```
SELECT *
FROM (
  SELECT
    p.categoria,
    p.nome_produto,
    ROUND(SUM(v.quantidade * v.preco_unitario), 2) AS receita,
    ROW_NUMBER() OVER (
      PARTITION BY p.categoria
      ORDER BY SUM(v.quantidade * v.preco_unitario) DESC
    ) AS ranking
  FROM ecommerce.bronze.vendas v
  JOIN ecommerce.bronze.produtos p ON v.id_produto = p.id_produto
  GROUP BY p.categoria, p.nome_produto
)
WHERE ranking <= 2
ORDER BY categoria;
```

RESULTADO

CATEGORIA	PRODUTO	RECEITA	RANKING
Acessórios	Necessaire	63.668,43	1
Acessórios	Cinto de Couro	21.506,77	2
Beleza	Sabonete Facial	11.268,99	2
Beleza	Condicionador Hidratante	31.587,75	1
Casa	Persiana Vertical	46.897,53	1
Casa	Jogo de Toalhas	17.528,83	2

SUM() OVER (): quanto cada canal representa

Sem `PARTITION BY`, a janela é a tabela inteira. Isso dá o total geral, útil como denominador para calcular percentuais.

CONSULTA

```
SELECT
  canal_venda,
  ROUND(SUM(quantidade * preco_unitario), 2) AS receita,
  ROUND(SUM(quantidade * preco_unitario) * 100
    / SUM(SUM(quantidade * preco_unitario)) OVER (), 1) AS pct
FROM ecommerce.bronze.vendas
GROUP BY canal_venda;
```

RESULTADO

CANAL	RECEITA	% DA RECEITA
loja_fisica	268.591,07	27,60
ecommerce	705.486,21	72,40

LAG: comparando com o dia anterior

CONSULTA

```
SELECT
  dia,
  receita,
  LAG(receita) OVER (ORDER BY dia) AS receita_dia_anterior,
  ROUND((receita - LAG(receita) OVER (ORDER BY dia)) * 100
        / LAG(receita) OVER (ORDER BY dia), 1) AS variacao_pct
FROM (
  SELECT DATE(data_venda) AS dia,
         ROUND(SUM(quantidade * preco_unitario), 2) AS receita
  FROM ecommerce.bronze.vendas
  GROUP BY DATE(data_venda)
)
ORDER BY dia;
```

RESULTADO

DIA	RECEITA	DIA ANTERIOR	VARIAÇÃO %
13/12/2025	27.980,74		
14/12/2025	40.740,48	27.980,74	45,60
15/12/2025	36.409,09	40.740,48	-10,60
16/12/2025	30.794,21	36.409,09	-15,40
17/12/2025	38.109,98	30.794,21	23,80

A primeira linha vem vazia, e isso é esperado: não existe dia anterior ao primeiro dia da base.

CAPÍTULO 19

Bônus: organizando consultas grandes

Subconsulta e CTE (WITH).

Quando a consulta cresce, existem duas formas de quebrá-la em pedaços.

Subconsulta

Uma consulta dentro da outra, entre parênteses. É o que usamos no capítulo anterior para filtrar pelo ranking. Funciona, mas de dentro para fora, o que dificulta a leitura.

CTE: WITH nome AS (...)

A **CTE** nomeia cada etapa e deixa a consulta legível de cima para baixo. Mesmo resultado, muito mais claro:

CONSULTA

```
WITH receita_por_produto AS (  
  SELECT  
    p.categoria,  
    p.nome_produto,  
    ROUND(SUM(v.quantidade * v.preco_unitario), 2) AS receita  
  FROM ecommerce.bronze.vendas v  
  JOIN ecommerce.bronze.produtos p ON v.id_produto = p.id_produto  
  GROUP BY p.categoria, p.nome_produto  
)  
,  
ranking AS (  
  SELECT  
    *,  
    ROW_NUMBER() OVER (PARTITION BY categoria ORDER BY receita DESC) AS posicao  
  FROM receita_por_produto  
)  
SELECT categoria, nome_produto, receita  
FROM ranking  
WHERE posicao <= 3  
ORDER BY categoria, posicao;
```

LEIA DE CIMA PARA BAIXO

Cada bloco do **WITH** é uma etapa com nome. Você monta o raciocínio em pedaços pequenos, testa cada um separado e junta no fim. Na Aula 2 essa ideia vira a arquitetura de camadas do pipeline.

CAPÍTULO 20

Por que tabela e não arquivo

Schema, histórico e viagem no tempo com Delta.

Quando você subiu o CSV, o Databricks não guardou o arquivo: ele criou uma **tabela Delta**. A diferença aparece em três situações.

1. A tabela tem contrato

Cada coluna tem um tipo, e a tabela recusa o que não se encaixa.

CONSULTA

```
DESCRIBE TABLE ecommerce.bronze.vendas;
```

RESULTADO

COLUNA	TIPO
id_venda	string
data_venda	timestamp
id_cliente	string
id_produto	string
canal_venda	string
quantidade	bigint
preco_unitario	double

Tente inserir uma venda com a quantidade escrita por extenso:

CONSULTA

```
INSERT INTO ecommerce.bronze.vendas
(id_venda, data_venda, id_cliente, id_produto, canal_venda, quantidade, preco_unitario)
VALUES ('sal_teste', current_timestamp(), 'cus_teste', 'prd_teste', 'ecommerce', 'duas',
99.90);
```

CAST_INVALID_INPUT: O ERRO QUE VOCÊ QUER VER

The value 'duas' of the type "STRING" cannot be cast to "INT". A tabela recusou o dado errado. Em um CSV, esse valor entraria em silêncio e quebraria o relatório de alguém no mês seguinte.

2. A tabela lembra do passado

Toda alteração vira uma **versão**. Faça uma mudança e olhe o histórico:

CONSULTA

```
UPDATE ecommerce.bronze.vendas
SET canal_venda = 'ECOMMERCE'
WHERE canal_venda = 'ecommerce';

DESCRIBE HISTORY ecommerce.bronze.vendas;
```

version	operation	operationParameters
1	UPDATE	predicate = canal_venda = 'ecommerce'
0	CREATE TABLE AS SELECT	o upload do CSV

3. Dá para voltar no tempo

Ops, padronizamos o canal errado. Em um arquivo, você teria perdido o original. Aqui, basta consultar a versão anterior (**time travel**) e restaurá-la:

CONSULTA

```
SELECT canal_venda, COUNT(*) AS vendas
FROM ecommerce.bronze.vendas VERSION AS OF 0
GROUP BY canal_venda;

RESTORE TABLE ecommerce.bronze.vendas TO VERSION AS OF 0;
```

O QUE ISSO SIGNIFICA NA PRÁTICA

Schema, histórico e transações são o que transformam um monte de arquivos em uma **plataforma de dados**. É a diferença entre “tenho uma planilha” e “tenho um dado em que a empresa confia”.

CAPÍTULO 21

O dashboard

Onde os diretores veem tudo sem abrir um notebook.

As respostas já existem. Faltava colocá-las onde os diretores enxergam. Um dashboard do Databricks tem duas peças:

Peça	O que é
Dataset	Uma consulta SQL, igual às que você escreveu hoje
Widget	O que aparece na tela: indicador, gráfico, tabela ou filtro

Montando, passo a passo

1. Menu lateral **Dashboards** → **Create dashboard**. Renomeie para *Diretoria E-commerce*.
2. Aba **Data** → **Create from SQL**: cole uma consulta e dê um nome ao dataset. Repita para cada pergunta (vendas, produtos, clientes, competitividade).
3. Volte ao **Canvas** e crie três páginas: Vendas, Clientes e Pricing.
4. Em cada página, arraste os widgets e escolha o dataset e as colunas.
5. Clique em **Publish** e compartilhe o link.

O que colocar em cada página

Página	Widgets sugeridos
Vendas	Indicadores de receita, vendas e ticket médio · linha de receita por dia · barras por categoria · tabela com os 10 produtos que mais faturam
Clientes	Indicadores de clientes e receita média · barras de receita por estado · tabela com o ranking de clientes
Pricing	Indicadores de produtos monitorados e de “mais caros que todos” · barras de diferença média por categoria · tabela de alerta

ATALHO

O repositório traz o dashboard pronto em [aula-01-sql-dashboard/dashboard/](#). Em **Dashboards** → **Import dashboard from file** você importa e publica em dois minutos. Vale montar do zero pelo menos uma vez, para entender as peças.

CAPÍTULO 22

Erros comuns e como ler a mensagem

Errar faz parte. O importante é entender o que o banco está dizendo.

Mensagens de erro no Databricks começam com um código em maiúsculas. Esse código é a parte mais útil: é ele que diz o que aconteceu.

Mensagem	O que significa	Como resolver
<code>TABLE_OR_VIEW_NOT_FOUND</code>	A tabela não existe com esse nome	Confira o endereço completo (<code>ecommerce.bronze.vendas</code>) e se o upload foi para o schema certo
<code>UNRESOLVED_COLUMN</code>	A coluna não existe	Provável erro de digitação; rode <code>DESCRIBE TABLE</code> para ver os nomes
<code>MISSING_AGGREGATION</code>	Coluna no <code>SELECT</code> fora do <code>GROUP BY</code>	Aplique a regra de ouro do capítulo 12
<code>CAST_INVALID_INPUT</code>	Valor incompatível com o tipo da coluna	Texto em coluna numérica; corrija o valor
<code>PARSE_SYNTAX_ERROR</code>	Erro de escrita	Vírgula faltando, aspas abertas ou ordem das cláusulas trocada
<code>DIVIDE_BY_ZERO</code>	Divisão por zero	Proteja com <code>NULLIF(divisor, 0)</code>
<code>AMBIGUOUS_REFERENCE</code>	Duas tabelas têm a mesma coluna	Use o apelido: <code>v.id_produto</code>

Três hábitos que evitam a maioria dos erros

- **Construa aos poucos.** Rode `SELECT *` primeiro, depois acrescente uma cláusula por vez. Se quebrou, foi a última coisa que você escreveu.
- **Confira os totais.** Principalmente depois de um `JOIN`: compare com o total sem `JOIN`.
- **Leia o fim da mensagem.** O Databricks costuma mostrar a linha e a posição exata do problema no fim do erro.

CONSULTAR NÃO ESTRAGA NADA

`SELECT` só lê. Não tem como quebrar a base rodando consulta. Erre à vontade: é assim que se aprende SQL.

CAPÍTULO 23

Exercícios

Dezesseis perguntas para praticar. As respostas estão no fim.

Nível 1 · ler dados

1. Mostre nome, categoria e preço de todos os produtos, do mais caro para o mais barato.
2. Quais são as 10 vendas com maior quantidade?
3. Quantas categorias diferentes existem no catálogo?
4. Quais produtos custam entre R\$ 100 e R\$ 500 e são da categoria Games?

Nível 2 · agregar

1. Qual o preço médio, mínimo e máximo do catálogo?
2. Quantos produtos existem em cada categoria?
3. Qual a receita total por dia da semana? (Dica: `DATE_FORMAT(data_venda, 'EEEE')`.)
4. Em qual hora do dia acontecem mais vendas?
5. Quais categorias têm mais de 20 produtos cadastrados?

Nível 3 · juntar tabelas

1. Qual marca gera mais receita?
2. Qual o ticket médio de cada estado?
3. Quais produtos nunca foram vendidos?
4. Qual concorrente tem o menor preço médio?
5. Quantas vendas cada cliente do estado de SP fez?

Nível 4 · desafio

1. Quantos clientes são VIP, TOP_TIER e REGULAR, e quanto cada segmento representa da receita?
2. Quais são os 3 produtos mais vendidos de cada categoria?

COMO PRATICAR

Não tente acertar de primeira. Escreva uma cláusula, rode, veja o resultado e acrescente a próxima. É assim que analista experiente trabalha.

Respostas dos exercícios

Compare com a sua consulta. Existe mais de um caminho certo.

Nível 1

CONSULTA

```
-- 1
SELECT nome_produto, categoria, preco_atual
FROM ecommerce.bronze.produtos
ORDER BY preco_atual DESC;

-- 2
SELECT id_venda, quantidade, preco_unitario
FROM ecommerce.bronze.vendas
ORDER BY quantidade DESC
LIMIT 10;

-- 3
SELECT COUNT(DISTINCT categoria) AS categorias
FROM ecommerce.bronze.produtos;

-- 4
SELECT nome_produto, preco_atual
FROM ecommerce.bronze.produtos
WHERE categoria = 'Games'
AND preco_atual BETWEEN 100 AND 500;
```

Nível 2

CONSULTA

```
-- 5
SELECT ROUND(AVG(preco_atual), 2) AS media,
       MIN(preco_atual) AS minimo,
       MAX(preco_atual) AS maximo
FROM ecommerce.bronze.produtos;

-- 6
SELECT categoria, COUNT(*) AS produtos
FROM ecommerce.bronze.produtos
GROUP BY categoria
ORDER BY produtos DESC;

-- 7
SELECT DATE_FORMAT(data_venda, 'EEEE') AS dia_semana,
       ROUND(SUM(quantidade * preco_unitario), 2) AS receita
FROM ecommerce.bronze.vendas
GROUP BY DATE_FORMAT(data_venda, 'EEEE')
ORDER BY receita DESC;

-- 8
SELECT HOUR(data_venda) AS hora, COUNT(*) AS vendas
FROM ecommerce.bronze.vendas
GROUP BY HOUR(data_venda)
ORDER BY vendas DESC;

-- 9
SELECT categoria, COUNT(*) AS produtos
FROM ecommerce.bronze.produtos
GROUP BY categoria
HAVING COUNT(*) > 20;
```

Nível 3

CONSULTA

```
-- 10
SELECT p.marca, ROUND(SUM(v.quantidade * v.preco_unitario), 2) AS receita
FROM ecommerce.bronze.vendas v
JOIN ecommerce.bronze.produtos p ON v.id_produto = p.id_produto
GROUP BY p.marca
ORDER BY receita DESC;

-- 11
SELECT c.estado, ROUND(AVG(v.quantidade * v.preco_unitario), 2) AS ticket_medio
FROM ecommerce.bronze.vendas v
JOIN ecommerce.bronze.clientes c ON v.id_cliente = c.id_cliente
GROUP BY c.estado
ORDER BY ticket_medio DESC;

-- 12
SELECT p.nome_produto, p.categoria
FROM ecommerce.bronze.produtos p
LEFT JOIN ecommerce.bronze.vendas v ON p.id_produto = v.id_produto
WHERE v.id_venda IS NULL;

-- 13
SELECT nome_concorrente, ROUND(AVG(preco_concorrente), 2) AS preco_medio
FROM ecommerce.bronze.preco_competidores
GROUP BY nome_concorrente
ORDER BY preco_medio;

-- 14
SELECT c.nome_cliente, COUNT(*) AS compras
FROM ecommerce.bronze.vendas v
JOIN ecommerce.bronze.clientes c ON v.id_cliente = c.id_cliente
WHERE c.estado = 'SP'
GROUP BY c.nome_cliente
ORDER BY compras DESC;
```

```
-- 15
WITH por_cliente AS (
  SELECT c.id_cliente,
         SUM(v.quantidade * v.preco_unitario) AS receita
  FROM ecommerce.bronze.vendas v
  JOIN ecommerce.bronze.clientes c ON v.id_cliente = c.id_cliente
  GROUP BY c.id_cliente
)
SELECT
  CASE WHEN receita >= 22000 THEN 'VIP'
        WHEN receita >= 17000 THEN 'TOP_TIER'
        ELSE 'REGULAR' END AS segmento,
  COUNT(*) AS clientes,
  ROUND(SUM(receita), 2) AS receita,
  ROUND(SUM(receita) * 100 / SUM(SUM(receita)) OVER (), 1) AS pct
FROM por_cliente
GROUP BY 1
ORDER BY receita DESC;

-- 16
SELECT * FROM (
  SELECT p.categoria, p.nome_produto,
         SUM(v.quantidade) AS itens,
         ROW_NUMBER() OVER (PARTITION BY p.categoria
                             ORDER BY SUM(v.quantidade) DESC) AS posicao
  FROM ecommerce.bronze.vendas v
  JOIN ecommerce.bronze.produtos p ON v.id_produto = p.id_produto
  GROUP BY p.categoria, p.nome_produto
)
WHERE posicao <= 3
ORDER BY categoria, posicao;
```

Colinha

Uma página para deixar aberta enquanto você escreve SQL.

A consulta completa

ESTRUTURA

```
SELECT  colunas, contas AS apelido
FROM    catalogo.schema.tabela apelido
JOIN    outra_tabela a2 ON apelido.chave = a2.chave
WHERE   filtro de linhas
GROUP BY colunas não agregadas
HAVING  filtro sobre agregações
ORDER BY coluna DESC
LIMIT  10;
```

Filtros (WHERE)

Escreva	Para
<code>coluna = 'texto'</code>	Igual (texto entre aspas simples)
<code>coluna > 100</code>	Maior que (número sem aspas)
<code>coluna BETWEEN 10 AND 20</code>	Entre dois valores
<code>coluna IN ('a', 'b')</code>	Está na lista
<code>coluna LIKE 'Tênis%'</code>	Começa com
<code>coluna IS NULL</code>	Está vazio
<code>cond1 AND cond2 / OR</code>	Combinar condições

Agregações

Escreva	Para
<code>COUNT(*)</code>	Contar linhas
<code>COUNT(DISTINCT coluna)</code>	Contar valores diferentes
<code>SUM(coluna)</code>	Somar
<code>AVG(coluna)</code>	Média
<code>MIN(coluna) / MAX(coluna)</code>	Menor / maior
<code>ROUND(valor, 2)</code>	Arredondar em 2 casas

Datas

Escreva	Para
<code>DATE(coluna)</code>	Só a data, sem hora
<code>HOUR(coluna)</code>	Hora (0 a 23)
<code>YEAR(coluna) / MONTH(coluna)</code>	Ano / mês
<code>DATE_FORMAT(coluna, 'EEEE')</code>	Dia da semana

Juntar tabelas

CONSULTA

```
FROM ecommerce.bronze.vendas v
JOIN ecommerce.bronze.produtos p      -- só o que casa nas duas
  ON v.id_produto = p.id_produto

FROM ecommerce.bronze.vendas v
LEFT JOIN ecommerce.bronze.produtos p  -- tudo da esquerda
  ON v.id_produto = p.id_produto
```

As quatro tabelas

Tabela	Colunas
<code>ecommerce.bronze.vendas</code>	id_venda, data_venda, id_cliente, id_produto, canal_venda, quantidade, preco_unitario
<code>ecommerce.bronze.produtos</code>	id_produto, nome_produto, categoria, marca, preco_atual, data_criacao
<code>ecommerce.bronze.clientes</code>	id_cliente, nome_cliente, estado, pais, data_cadastro
<code>ecommerce.bronze.preco_competidores</code>	id_produto, nome_concorrente, preco_concorrente, data_coleta

GUARDE ESTE NÚMERO

Receita total do período: **R\$ 974.077,28** em 3.020 vendas. Se a sua consulta devolver outro valor sem motivo, alguma coisa filtrou ou duplicou linhas.

Os próximos três dias

Onde este projeto chega.

Hoje você respondeu os diretores **na mão**: subiu os arquivos, escreveu as consultas e publicou o painel. Nos próximos dias, o mesmo projeto evolui.

Dia	Tema	A pergunta do dia
2	Python & Engenharia de Dados	De onde vieram esses dados, e como eles chegam sozinhos todo dia?
3	Claude Code & Engenharia de Dados	Como transformo isso em um projeto profissional, com Git, testes e deploy?
4	Genie	Como o diretor pergunta sozinho, em português, sem depender de mim?

Para chegar preparado na Aula 2

- Verifique sua conta do Databricks pelo LinkedIn quando ele pedir: sem isso o ambiente não acessa a internet, e a Aula 2 busca dados de uma API.
- Refaça pelo menos três consultas deste guia sem olhar a resposta.
- Rode os exercícios do capítulo 23.

UMA ÚLTIMA IDEIA

Você não aprendeu ferramentas hoje. Você aprendeu a transformar uma pergunta de negócio em uma consulta, e uma consulta em decisão. Cada query que você escrever daqui para frente deveria responder: **por que isso importa?**